fn sample_uniform_uint_below

Michael Shoemate

April 20, 2026

This document proves that the implementation of `sample_uniform_uint_below` in `mod.rs` at commit f5bb719 (outdated[1]) satisfies its definition.

`sample_uniform_uint_below` uses rejection sampling. In each round all bits of the integer are filled randomly, drawing an unsigned integer uniformly at random from a space of size 2^N. The algorithm returns the sample, modulo the upper bound, so long as the sample is not one of the first $2^N \mod$ `upper` integers.

0.1 Hoare Triple

Preconditions

Compiler-verified

- Generic `T` has traits `Integer + Unsigned + FromBytes<N>`.
- Const-generic `N` of type usize. By the definition of `FromBytes`, the compiler ensures this is the number of bytes in T.
- Argument `upper` must be of type `T`

User-verified `upper` > 0

Pseudocode

```
def sample_uniform_uint_below(upper: T) -> T:
    assert upper > 0
    reject_below = (T.MAX % upper + 1) % upper

    while True:
        sample = sample_from_uniform_bytes()
        if sample >= reject_below:
            return sample % upper
```

Postcondition

For any setting of the input parameter `upper` that satisfies the preconditions, `sample_uniform_uint_below` either

- raises an exception if there is a lack of system entropy,
- returns `out` where `out` is uniformly distributed between $[0, upper)$.

[1] See new changes with `git diff f5bb719..407179b7 rust/src/traits/samplers/uniform/mod.rs`

0.2 Proof

Proof. By the postcondition of `sample_from_uniform_bytes`, `sample` is drawn uniformly from the 2^N representable values of type `T`, namely $\{0, \ldots, \texttt{T.MAX}\}$.

Let $r = 2^N \mod \texttt{upper}$. Since `upper` is positive by the preconditions, r is well-defined and $\texttt{reject_below} = r$. The algorithm rejects exactly the first r values and accepts the remaining $2^N - r$ values. This is equivalent to the more familiar presentation that accepts the first $2^N - r$ values and rejects the final r values, because both acceptance regions have the same size and therefore contain the same number of representatives from each congruence class modulo `upper`. The implementation uses front rejection because $2^N = \texttt{T.MAX} + 1$ may not itself be representable in type `T`; however, r is representable, so testing whether `sample >= reject_below` avoids overflow. Because $2^N - r$ is a multiple of `upper`, the accepted values partition into congruence classes modulo `upper` of equal size. Therefore `sample % upper` is uniformly distributed on $[0, \texttt{upper})$ after conditioning on acceptance.
Therefore, for any positive value of `upper`, the function satisfies the postcondition. □